

A Neurosymbolic Approach to Natural Language Formalization and Verification

Chenyang An[1], Sam Bayless[1], Stefano Buliani[1], Darion Cassel[1], Byron Cook[1,2], Duncan Clough[1], Rémi Delmas[1], Nafi Diallo[1], Ferhat Erata[1], Nick Feng[1], Dimitra Giannakopoulou[1], Aman Goel[1], Aditya Gokhale[1], Joe Hendrix[1], Victor Heorhiadi[1], Marc Hudak[1], Dejan Jovanović[1], Andrew M. Kent[1], Benjamin Kiesl-Reiter[1(✉)], Jeffrey J. Kuna[1], Nadia Labai[1], Joseph Lilien[1], Divya Raghunathan[1], Zvonimir Rakamarić[1], Niloofar Razavi[1], Michael Tautschnig[1,4], Ali Torkamani[1], Nathaniel Weir[1], Michael W. Whalen[1], and Jianan Yao[3]

[1] Amazon Web Services, Seattle, USA
benkiesl@amazon.de
[2] University College London, London, UK
[3] University of Toronto, Toronto, Canada
[4] Queen Mary University of London, London, UK

Abstract. Large Language Models perform well at natural language interpretation and reasoning, but their lack of formal correctness guarantees limits their adoption in regulated industries like finance and healthcare that operate under strict policies. To address this limitation, we launched Automated Reasoning checks (ARc): a public service that (1) uses LLMs with optional human guidance to formalize natural language policies, allowing fine-grained control of the formalization process, and (2) uses inference-time autoformalization to validate logical correctness of natural language statements against those policies. ARc performs multiple redundant formalization steps at inference time, checking the formalizations for semantic equivalence. Our benchmarks show that ARc exceeds 99% soundness and achieves a near-zero false positive rate in identifying logical validity. Our approach produces auditable artifacts that substantiate the verification outcomes and can be used to improve the original text. ARc is the first commercial offering from a major cloud provider to integrate automated reasoning into a generative AI guardrail.

1 Introduction

The capabilities of Large Language Models (LLMs) continue to advance rapidly, demonstrating unprecedented improvements in coherence and analytical accuracy [20,39,44]. Despite these advances, their tendency to generate plausible but incorrect information (hallucinations, cf. [42]) remains a barrier to widespread adoption in regulated sectors. Industries such as healthcare, financial services, and legal practices have legal and regulatory obligations for accuracy and auditability that current LLM technology has yet to meet [9].

© The Author(s) 2026
E. Darulova et al. (Eds.): CAV 2026, LNCS 16683, pp. 601–617, 2026.
https://doi.org/10.1007/978-3-032-32526-6_28

Companies develop institutional policies to ensure compliance with laws and regulations. Such policies are typically captured in natural language (NL), defining rules, procedures, or guidelines. When deploying LLMs to answer questions about these policies, a key challenge emerges: can we develop *guardrails* to ensure that the LLM outputs are correct? Consider an airline implementing a chatbot to assist customer service representatives in navigating refund policies: if the chatbot incorrectly claims that a customer is eligible for a refund, this could lead to legal exposure and loss of customer trust.

An effective guardrail would enable representatives to rely on chatbot responses by ensuring that when it reports an answer as valid, it actually is. Inspired by the concept of soundness in logic, we define *soundness* in our context as $(1-r)$, where r is the rate of incorrect validity claims across all decisions. High soundness thus means that across all requests, incorrect approvals are rare. Following established practices in safety-critical systems, where reliability is often measured in "nines" (e.g., 99% = "two nines," 99.9% = "three nines"), we target soundness levels of at least 99%, and secondarily focus on recall to maximize the probability of accepting valid content. We also pursue actionable feedback that steers LLMs toward content that a conservative guardrail can accept.

A natural candidate for robust guardrails are symbolic reasoning systems, as they use formal logic to generate independently verifiable guarantees [30]. This aligns well with policy documents, which rely on logical, rule-like statements (e.g., "if a flight is canceled or ..., then passengers are entitled to a refund"). But symbolic methods struggle with interpreting natural language, triggering the development of neurosymbolic approaches that combine the NL processing capabilities of neural networks with the mathematical rigor of symbolic systems [12].

This paper presents Automated Reasoning checks (ARc), a neurosymbolic approach exceeding 99% soundness on datasets it was not trained on. High soundness is also reflected in metrics such as false positive rate and precision. In addition, ARc delivers explainable verdicts and provides actionable feedback for revising LLM outputs. ARc is the first commercial offering from a major cloud provider to integrate automated reasoning into a generative AI guardrail.

ARc operates through two complementary components. The first, called Policy Model Creator (PMC), combines LLMs with symbolic reasoning to translate NL policies into formal *policy models* expressed in logic. It begins with an autoformalization phase that generates an initial policy model. This is followed by an optional vetting phase where domain experts refine the policy model with assistance from the system. Vetting enables domain experts to resolve ambiguities and inconsistencies in the original documents, or to correct potential imprecisions from autoformalization. Policy model creation occurs offline; its computation cost will be amortized across subsequent verification tasks.

The second component, called Answer Verifier (AV), verifies NL content against policy models. The AV uses LLMs to translate NL content into individual logical claims in the scope of the policy model. Each claim is analyzed separately and assigned a verification result, together with detailed logical explanations and corrective guidance. To increase reliability, the AV uses multiple LLMs to

simultaneously formalize the same NL content, then uses symbolic reasoning to compare formalizations and assign confidence scores. The AV delivers auditable logical artifacts that substantiate the verification outcomes.

2 Related Work

Recent approaches use LLMs as judges to evaluate factual accuracy [16], though these rely on the same LLMs that are subject to inaccuracies to evaluate LLM-generated content. MiniCheck [35] provides efficient fact-checking by decomposing claims. RefChecker [13] introduces knowledge-centric verification against structured knowledge bases. SelfCheckGPT [22] leverages consistency across responses to detect hallucinations. FactCheck-GPT [38] provides comprehensive evaluation with fine-grained error categorization. While promising, these methods cannot provide formal guarantees. Our neurosymbolic framework verifies logical validity against formalized policies, achieving near-zero false positives.

Neurosymbolic systems combine LLMs (sometimes enhanced with Chain-of-Thought prompting [10,21,41]) with symbolic reasoning. They typically translate NL to formal representations that are then solved by external reasoners [5,26,27,31]. LINC [26] uses first-order logic with Prover9 [23]. Verus-LM [5] provides a multi-paradigm framework with IDP-Z3 [6]. SAT-LM [45] employs declarative prompting with SMT [25]. Logic-LM [27] supports multiple formalisms with self-refinement. Other approaches use custom DSLs [8] or Answer Set Programming [4,15,43].

These neurosymbolic systems formalize both the background knowledge and the query in a single step for each problem instance (or, in the case of [43], rely on hand-written background knowledge). In contrast, ARC autoformalizes both but separates them: the policy model can be vetted independently (through linting, symbolic test case generation, and human review) before being reused across many verifications. Regarding translation correctness, most of these systems offer only syntactic error detection and basic consistency checks (e.g., satisfiability of the formalized theory). LINC goes further by sampling multiple translations from a single model and applying majority voting over solver outcomes (Valid, Invalid, Unknown) rather than comparing translations at the logic level. ARC cross-checks per-query translations across diverse models using a notion of symbolic equivalence, with confidence thresholds indicating semantic agreement.

Autoformalization has been studied extensively in mathematics [17,33,37,40]. In contrast to ARC, these efforts typically assume that the background knowledge (the mathematical theory) has been pre-formalized, often by humans, and only the problem to be solved must be formalized by an LLM.

There is also a substantial body of work on dedicated formal languages for capturing policies and legislation, see, e.g., the Programming Languages and the Law (ProLaLa) workshops at POPL. Catala [24] is a notable example: a domain-specific language designed for systematic translation of statutory law into executable code, using defeasible reasoning to handle the general-case/exceptions logic common in legal texts. Such approaches capture laws directly in the dedicated language, whereas ARC autoformalizes existing NL policy documents into

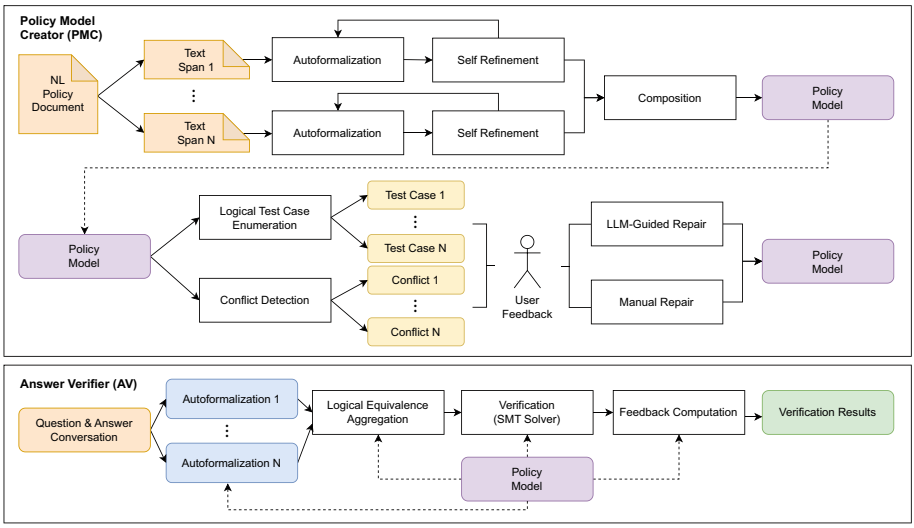


Fig. 1. End-to-end architecture of ARC

logic. Casadio et al. [7] propose a methodology for certifying the robustness of NLP models, showing how verification errors can stem from the NL representation layer. Their work on robustness guarantees for NL models is complementary to our redundant translation approach for mitigating translation errors.

3 Methodology

Figure 1 shows ARC's two main components: the PMC (Sect. 3.1) and the AV (Sect. 3.2). We illustrate our approach with an NL policy about park admission fees:

> **General admission**: The regular admission to the park is $50. The admission fee in the low season is 75% of the regular admission fee.
>
> **Discount**: Seniors (age greater than 65) qualify for a 40% discount. Whenever a discount applies, there will be a $10 flat discount processing fee.
>
> **Credit**: You can use credit for up to 50% of your final admission (1 credit for 1 dollar). However, if credits are used, then the discount rate is capped at 25%. You can purchase credit at a rate of $0.60 per credit. You can only purchase credit in increments of 5 (cost $3).
>
> **Tax**: A federal tax of 10% applies to the final expense.

Suppose a user asks "I am a senior and want to visit the park in the low season, and I have a budget of $35.40. Can I visit the park?", and we want to verify a chatbot's answer of "No, $35.40 is not enough."

ARC tackles the verification problem in two stages. In the first stage, the PMC autoformalizes the policy into a *policy model*: a set of logic rules, expressed

in SMT-LIB [3], together with a *schema* that defines variables with their types and NL descriptions. SMT-LIB is a standardized logic language that uses prefix notation, where operators precede their arguments; e.g., "if x, then y" is written as `(=> x y)`. Figure 2 shows snippets of the policy model. In the second stage, the AV autoformalizes the statement under validation into logic formulas (over the policy model schema), then uses an SMT solver to verify those formulas against the policy model. The result includes the logic translation (with a confidence score between 0 and 1), the validation result, and further feedback (see Fig. 3).

In our example, the result is *Satisfiable*: the claim (cannot visit the park) is consistent with the premises (the person is a senior, it is low season, and they have a fund of $35.40 available) but doesn't necessarily follow from them. AV provides two assignments as feedback: one showing a counter-example where admission is possible, and a case where the person cannot enter the park. The key difference is in the use of credits (*creditUnit* = *3* vs. *creditUnit* = *0*).
Explanation. The person needs to use $15 worth of credits for admission. The admission fee in the low season is $37.5 (75% of $50). After applying the 25% senior discount (capped at 25% because of credit use) and adding the $10 discount processing fee, the actual admission fee becomes $ 38.125. This fee can be paid by combining $15 of credit (cost: $9) with a $23.125 cash payment, for an expense of $ 32.125. After adding a federal tax of 10%, the final expense becomes $35.3375, which is within the budget of $35.40. Both Claude Sonnet 3.7

Variable	**Type**	**Description**
isLowSeason	Bool	Whether the admission day is in the low season
feeAfterDiscount	$\mathbb{R}$	Admission fee after discounts are applied but before tax

Rule 1: `(=> isLowSeason (= admissionFee (* 0.75 baseFee)))`

Rule 2: `(<= (* 2.0 customerCredits) feeAfterDiscount)`

Fig. 2. Snippets of policy model (top: variable schema; bottom: rules).

Translation (Confidence: 1.0)
P: `(and (= ageClass SENIOR) isLowSeason (= totalAdmissionFund 35.4))`
C: `(not isEntryAllowed)`
Result: *Satisfiable* (not *Valid*)
Counter-Example (shows C can be false):
creditUnit=3, customerCredits=15.0, creditDollarValue=9.0,
cashAmount=23.181, totalPaymentAvailable=38.181,
finalAdmissionFee=38.125, isEntryAllowed=true, ...
Satisfying Assignment (shows C can be true):
creditUnit=0, finalAdmissionFee=35.75, isEntryAllowed=false, ...

Fig. 3. Snippet of validation feedback.

and Opus 4.1 (with reasoning mode) incorrectly classified the answer as valid, providing plausible but flawed reasoning (see Figs. 10 and 11 in the extended version of the paper [2] for the full LLM-judge transcripts).

3.1 *Policy Model Creator (PMC)*

The PMC takes a policy document written in natural language and autoformalizes it into a policy model. It also provides an array of utilities to support users in policy vetting, described in detail in the *Policy Vetting* paragraph below.

Autoformalizing Policies. To handle the size and complexity of real-world policy documents in the face of known LLM reasoning limitations around context size and distractors [19,29], the PMC takes a divide-and-conquer approach to autoformalize documents into logic (see Fig. 1).

The PMC first splits the input document into a set of text spans. These are processed using an incremental, refinement-guided autoformalization procedure: A language model processes each span and identifies statements that express coherent, formalizable meaning. For each statement, the LLM translates the semantic content into a list of SMT-LIB datatypes, variables, and logical constraints (*rules*). The LLM's context maintains existing declarations within a span to avoid duplicated or conflicting declarations. The complete formalization of a span is what we call a *policy unit*. If this process introduces an error (e.g., malformed syntax), we provide the invalid declarations and their failure causes to the LLM for repair in a refinement loop.

After the PMC has formalized all text spans, it composes the policy units into a single policy model. The PMC generates textual embeddings of variables and clusters them using cosine similarity. Variables within a cluster are unified, while variables that share the same name but are not clustered are renamed. Consistent replacement of original variables with unified variables is performed for the rules of each policy unit, then the rules are aggregated, dropping syntactic duplicates. The resulting policy model is a structured representation of the document, consisting of three fields: datatypes, variables, and rules. Each variable is associated with an NL description that explains its meaning in terms of the source document, as shown in Fig. 2. This initial policy model is then vetted, as described in Sect. 3.1. The relationship between document size and formalized policy size is reported in the extended version of the paper [2], Appendix A.1.

PMC Policy Vetting. The initial policy model might contain errors and omissions. Additionally, as shown in Sect. 4.2, NL policies often contain ambiguities that only subject matter experts can resolve. We therefore provide users with several methods for vetting of their policy models: linting, inspection, and testing (both manual and automatic). We also develop automated repair approaches around these vetting methods.

Linting. A linter for our policy models that checks integrity and consistency properties beyond the simple malformedness errors caught during autoformalization.

We perform a mix of syntax-based and semantic checks: we detect unused variables and types, contradictory rules, and disjoint rule sets. The list of warnings is shown to the user, who can address them directly (e.g., by deleting an unused variable) or through more detailed policy inspection and repair.

Inspection. Manual inspection allows users to review the generated policy model, similar to code review in software development. We assist users by providing two views of the rules for inspection: SMT-LIB for experts and structured English for non-experts. Structured English is generated using templates (like "if ... then ...") to avoid potential hallucinations from LLMs. Users also can provide NL feedback on the policy, which triggers an automatic LLM-based policy repair step that adjusts the policy model. Manual inspection provides strong correctness guarantees when all rules are carefully reviewed, but it can be challenging with large numbers of complex rules that have intricate interactions. The PMC therefore also provides testing as an additional policy vetting methodology.

Testing. Testing provides a systematic way to validate policy models through examples. Similar to unit tests, test cases in the PMC are either NL question-answer pairs or simple NL statements with their expected outcomes provided by the user (e.g., valid, invalid). Test cases can either be provided manually by users or generated automatically. For manually provided test cases, the PMC "executes" them by running the AV to compute the findings; a mismatch indicates an error in the policy model or in the AV translation. The PMC also offers automatic, symbolic test-case generation that leverages an SMT solver to systematically explore the state space of the policy model. Since such test cases are generated symbolically, each comes with its provably-correct actual finding, so a mismatch isolates the error to the policy model. Either way, the supplied information (e.g., rules justifying the result) helps users diagnose and repair.

3.2 *Answer Verifier (AV)*

The AV uses LLMs to translate natural language (i.e., question-answer pairs) into a list of premise-conclusion pairs, where premises (P) and conclusions (C) are expressed in the policy model's schema. For example, "Since you have at least \$50, you can enter the park" translates to premise ($\geq$ *totalAdmissionFund* 50) and conclusion *isEntryAllowed* which represent the contextual facts and logical consequences, respectively.

To increase translation confidence, the AV *redundantly translates* the NL statement using k LLMs (Algorithm 1), then compares the resulting premise-conclusion pairs semantically using an SMT solver to estimate a confidence score for each pair. Intuitively, the confidence score of a premise-conclusion pair $\langle P, C \rangle$ is the proportion of the k translations that non-vacuously entail the implication $P \Rightarrow C$ (i.e., entail it without rendering P contradictory). For example, consider the text from Sect. 3 and the policy model in Fig. 2. Redundant translation with three LLMs produces three identical premise-conclusion pairs, thus producing the results in Fig. 3 with confidence score 3/3 (1.0). If one LLM instead produced a pair with a different conclusion (`isEntryAllowed`), AV would return two distinct

Algorithm 1. AV Redundant Translation

procedure RedundantTranslation(msg, $policy$, $LLMs$)
  $F \leftarrow \emptyset$; $Ts \leftarrow [Translate(msg, policy, llm) \mid llm \in LLMs]$
  for each $\langle P, C \rangle$ in every $T \in Ts$ **do**
    F.add($\langle P, C, cf \rangle$) where $cf = |\{T' \in Ts \mid T' \models (P \Rightarrow C) \wedge T' \not\models \neg P\}|/|Ts|$
  return F

pairs: the original from Fig. 3 with confidence 2/3, and the alternative with confidence 1/3.

Validation Feedback. After translation, the AV uses an SMT solver to validate each claim $\langle P, C, conf \rangle$ against $\mathcal{M}$ while producing logically grounded feedback given the following precedence order:

Finding	Condition
TooComplex	Text or SMT-LIB translation exceeds token limits
TranslationAmbiguous	Confidence below threshold (default: 3/3)
NoTranslations	LLM cannot translate text to policy model vocabulary
Impossible	$\mathcal{M} \models \neg P$ (premises contradict policy model)
Invalid	$\mathcal{M} \wedge P \models \neg C$ (conclusion is inconsistent)
Valid	$\mathcal{M} \wedge P \models C$ (conclusion is entailed)
Satisfiable	$\mathcal{M} \wedge P \not\models C$ and $\mathcal{M} \wedge P \not\models \neg C$ (consistent, not entailed)

For *Impossible*, *Invalid*, and *Valid* findings, the feedback includes relevant rules from the policy model extracted from the SMT solver, providing sufficient information for independent verification via theorem prover. For *Satisfiable* findings, the feedback returns assignments ("scenarios") demonstrating how the answer can be correct or wrong, thus providing hints on how the premises could be extended to make the conclusion valid (see Fig. 3, for example, where the use of credits is a differentiator between the scenarios). For *TranslationAmbiguous* findings, the feedback presents two differing translations, together with an assignment that is satisfiable in one translation but not in the other. *NoTranslations* findings return the untranslatable text segments. Logic warnings are surfaced if premises or conclusions are always true or false irrespective of policy rules.

4 Empirical Evaluation

In order to understand ARc's effectiveness as a guardrail, and the impact of our design choices, we evaluate ARc, and more specifically the AV, around the following research questions (RQs):

RQ1 (Reliability of Validating Logical Accuracy): How reliably does ARc validate logical accuracy compared to alternative baselines?

RQ2 (Impact of Redundant Translation): How does redundant translation (Sect. 3.2) impact ARc's performance?
RQ3 (Effectiveness of ARc's feedback): Is ARc's feedback effective in driving improvement of LLM outputs?

Metrics. We frame logical accuracy detection as a binary classification problem: decide whether NL statements are *Valid* or not.

To target high-stakes applications, our primary objective is to eliminate false positives across the entire pipeline. In this context, rejecting borderline cases (*not-valid*) is favorable: such outputs can either be refined by the answer-generating LLM using ARc feedback, or escalated to human experts. We thus define *soundness* as $1 - \frac{\#\text{False Positives}}{\#\text{Samples}}$, measuring the rate at which incorrect approvals occur across all decisions.

Table 1. Comparison of logical accuracy detection performance on ConditionalQA-logic [32]. The columns show soundness (S), false-positive rate (FPR), precision (Pr), recall (Re), F1 score (F1), accuracy (Ac), counts of true/false positives/negatives (TP/FP/TN/FN), and error count (Error#). Approaches meeting soundness threshold are highlighted, as well as best and worst values for other metrics.

Method	S ↑	FPR ↓	Pr ↑	Re ↑	F1 ↑	Ac ↑	TP ↑	FP ↓	TN ↑	FN ↓	Error# ↓
ARc (#3-ensemble, threshold=3/3)	99.4	1.8	94.4	14.9	25.8	42.4	169	10	548	962	5
ARc (#3-ensemble, threshold=2/3)	99.2	2.5	93.2	16.9	28.6	43.5	191	14	544	940	9
ARc (without redundant translation)	98.4	4.8	92.2	28.0	43.0	50.2	317	27	531	814	1
LLMaJ (#3-ensemble, threshold=3/3)	98.2	5.4	92.4	32.4	47.9	52.9	366	30	528	765	-
LLMaJ (#3-ensemble, threshold=2/3)	97.9	6.3	92.1	36.3	52.0	55.2	410	35	523	721	-
LLMaJ (1x Sonnet4.5)	97.9	6.3	92.0	35.4	51.1	54.6	400	35	523	731	-
LLMaJ (1x Sonnet4.5 w/ extended thinking)	97.1	8.8	92.7	55.0	69.0	67.0	622	49	509	509	-
FG Implicit span-level [16]	95.0	15.2	90.4	70.6	79.2	75.3	798	85	473	333	-
FG JSON [16]	94.6	16.5	89.7	70.5	78.9	74.8	797	92	466	334	-
FG Response-level [16]	83.8	49.1	78.3	**87.6**	82.7	75.5	**991**	274	284	**140**	-
MiniCheck [18]	88.5	34.9	83.0	84.4	83.7	78.0	954	195	363	177	-
RefChecker [13]	91.9	24.6	86.3	76.0	80.8	75.8	860	137	421	271	-
SelfCheckGPT [22]	93.0	21.1	89.2	86.3	**87.7**	**83.8**	976	118	440	155	-
Logic-LM [27]	98.0	5.9	86.4	18.6	30.6	43.5	210	33	525	921	1149
Proof of Thought [8]	88.8	33.9	81.2	72.2	76.4	70.2	817	189	369	314	**0**
LINC [26]	**99.9**	**0.2**	**98.7**	6.6	12.4	37.4	75	**1**	**557**	1056	696

A natural alternative to soundness would be precision. The two metrics differ in perspective: precision estimates a posterior ("Given a request was classified as *Valid*, how likely is that verdict to be correct?"), while soundness estimates a prior ("How likely is a request to receive an incorrect *Valid* verdict?"). Based on customer feedback, we prioritized the prior perspective: ARc's customers are service operators deploying chatbots across thousands of daily interactions, where the relevant operational question is how likely any given request is to be incorrectly approved. Like any soundness notion, our soundness rewards abstention

and is incomplete on its own, so we use recall to complement it. As mentioned in Sect. 1, we target soundness levels of at least 99%.

In addition to soundness, we use standard classification metrics (precision, recall, F1, accuracy), treating *Valid* as the positive class and all others as negative. When comparing alternative methods, *Valid* recall is used as a tie-breaker under the requirement of maintaining high soundness.

4.1 Evaluation of Logical Accuracy Validation

Dataset. Several reasoning benchmarks (e.g., FOLIO [11], ProofWriter [34], and LogicNLI [36]) test logical inference, but our focus is on validating whether NL answers comply with formalized policy documents. We therefore focus on the ConditionalQA dataset [32] because it is well-aligned with our task, featuring: 1) questions that require compositional logical reasoning, 2) variety of questions (yes/no, multiple answers, not-answerable), and 3) human annotated answers.

We enrich the ConditionalQA dev dataset (391 labeled QAs over 59 source documents) with several types of "not valid" examples beyond its original "valid" / "not_answerable" classification. The resulting set includes the following categories: *Valid* (logically correct), *Invalid* (incorrect due to wrong conditions), *Satisfiable* (missing necessary conditions), *Impossible* (contradictory conditions), and *NoTranslations* (content that cannot be formalized, originally classified as not-answerable). These categories were created by systematically manipulating the conditional structure of original answers: removing conditions (*Valid* → *Satisfiable*), negating the claim (*Valid* → *Invalid*), or merging contradictory conditions (*Valid* → *Impossible*). The extended dataset (ConditionalQA-logic) contains 377 *Valid* examples and 186 examples that are not *Valid* (112 *Invalid*, 56 *Satisfiable*, 4 *Impossible*, and 14 *NoTranslations*).

Our goal in this section is to evaluate the AV's soundness in an end-to-end setting with automatically generated, unvetted policy models. The policy models used for Table 1 were autoformalized by the PMC and not manually refined. Section 4.2 provides initial evidence that human vetting can further improve both soundness and recall.

RQ1: Reliability of Validating Logical Accuracy. Table 1 reports on the comparison of ARc against alternative methods: LLM-as-Judge (LLMaJ) with different prompting strategies, FACTS Grounding (FG) [16], fine-grained hallucination detection methods [13,18,22,35], and other neurosymbolic approaches [8,26,27]. This evaluation focuses on the ability of each approach to predict validation labels for QA pairs about given NL policy documents. We evaluate each of the 563 benchmark instances three times to account for nondeterminism in LLM outputs, yielding 1689 total decisions.

We first examine the approaches that meet our soundness threshold, which are only ARc and LINC [26]. Between the two, ARc has a higher recall (14.9% vs 6.6%) and significantly fewer errors (5 vs 696 out of 1689 instances), where errors are instances in which the system was unable to formalize the QA contents and thus could not produce a verdict. Second, we compare ARc to others that did not achieve the required soundness threshold of 99%. ARc's reliability comes

with lower recall (14.9% for the configuration with soundness over 99%), but the tradeoff is intentional: in safety-critical domains, false approvals are far more costly than false rejections. Among the methods below the soundness threshold, LLMaJ (#3-ensemble, threshold = 3/3) comes closest at 98.2% soundness with precision similar to ARC's (92.4% vs 94.4%), but produces 3 times as many false positives (30 vs 10, or 5.4% vs 1.8% FPR). In contrast, FG Response-level has the highest recall of 87.6%, but this comes at the cost of soundness dropping to just 83.8%, the second-lowest of all methods.

RQ2: Impact of Redundant Translation. ARC uses redundant translation (Algorithm 1) to increase confidence in NL-to-logic translations. Comparing the first 3 rows of Table 1: soundness rises from 98.4% to 99.4% and FPR drops from 4.8% to 1.8%, at the cost of reduced recall (28.0% to 14.9%). Lowering the confidence threshold from 3/3 to 2/3 recovers recall to 16.9%, with soundness dropping to 99.2%.

ARC's recall is low compared to the other techniques in Table 1, meaning that in many cases it returns results other than *Valid* for content that should be *Valid*. ARC is intended to be deployed as a guardrail for chatbots in regulated industries, where an incorrect approval (e.g., 'you qualify at 0% interest') causes real harm at scale. In this setting, a non-*Valid* verdict is not a failure: the system can revise the answer using ARC's feedback (see RQ3) or route to a customer support agent. Low recall thus means more deferrals, not more failures, and at 99.4% soundness, *Valid* verdicts carry strong enough assurance to reduce the review burden. In Sect. 4.2, we demonstrate that three rounds of revision with ARC's feedback can drive the rate of *Valid* answers from 9% to 46% on a real-world policy.

4.2 Refining Real-World Policy Models and Answers

To understand the applicability of ARC to real-world policies, we collected customer-facing policy documents from six different businesses, refined with human-in-the-loop vetting. We use these policies to evaluate iterated self-refinement of LLM answers using ARC feedback (*RQ3*); Appendix A.1 in the extended version of the paper [2] additionally shares one policy as a case study of the manual refinement process itself (soundness: 96.8% → 100%, recall: 25% → 45.5%).

RQ3: Effectiveness of ARC*'s Feedback.* In a real-world setting, users may deploy LLM-based chatbots to answer questions about policies,

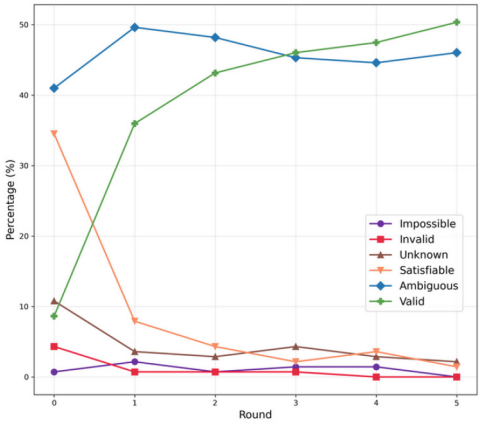


Fig. 4. ARC validation finding distribution after k iterations of answer revision using ARC feedback. At $k = 0$, we plot the finding distribution before any revisions.

requiring assurance of answer correctness. The formally-grounded feedback that ARC provides (Sect. 3.2) can be used for automated answer revision: given a non-*Valid* verdict, an LLM iteratively revises its answer using ARC's feedback. Figure 4 shows how the relative percentages of each finding type evolve as an LLM iteratively revises AV-judged non-*Valid* answers using ARC feedback (#3-ensemble, threshold = 3/3). After just three iterations, the LLM goes from 9% to 46% *Valid* answers. Primarily, this comes from a sharp reduction in AV-judged *Satisfiable* answers (where the answer could be true or false depending on context not provided in the question or answer). By providing logically-derived scenarios showing when the answer is true and when it is false, ARC enables the LLM to effectively revise these into *Valid* answers. ARC's feedback is less effective in revising *TranslationAmbiguous* and *NoTranslations* answers. Our analysis shows that in these cases revising the policy model is more effective; e.g., the policy model could be missing variables, leading to a failure to formalize the answer, or the policy model could have variables that overlap in meaning, leading to a failure to generate a consistent formalization of the answer. Average iterations to *Valid* and success rate per finding type are reported in the extended version of the paper [2], Table 5.

5 Conclusions, Limitations, and Future Work

We presented ARC, a neurosymbolic guardrail that exceeds 99% soundness when validating LLM answers against NL policies. This soundness comes at the cost of recall, a tradeoff we believe appropriate for regulated industries. Since its launch, ARC has been adopted across industries, including responsible AI in education [28], financial AI agents [14], and logistics operations [1]. Soundness of ARC heavily depends on the quality of the policy model that it uses for validation. For this reason, ARC enables human oversight. As we have shown, ARC provides meaningful feedback to aid automated answer revision, resulting in an increase of *Valid* answers. Despite ARC's high soundness, there are limitations:

- *Document types*: Policies with numerical tables, cross-references, or implicit assumptions can be challenging to formalize without human vetting.
- *Autoformalization challenges*: Subtle issues like ambiguous pronouns or implicit temporal scoping can lead to incorrect formalizations.
- *Computational cost*: Redundant translation requires multiple (3) LLM calls, resulting in average 5–15 s latency and increased API cost per Q/A validation with our current implementation.
- *Human effort*: The investment for human vetting of policies, while amortized over time, remains a significant upfront cost, especially for large documents.
- *Dataset representativeness*: The mechanically mutated evaluation examples (removing conditions, negating claims, merging contradictory conditions) may not fully capture the kinds of errors real LLMs produce in practice. Evaluating on naturally occurring LLM failures is a direction for future work.

Future work includes exploring automatic and confidence-aware focused vetting, fine-tuned translation models for improving accuracy and latency/costs, and improved logical formalisms to address current limitations. Our approach directly benefits from advances in LLMs and generative AI techniques: as models improve, their ability to formalize natural language to logic will too. We are confident ARc will inherit these improvements while maintaining the mathematical guarantees provided by symbolic reasoning.

Data Availability Statement. ARc is publicly available as part of Amazon Bedrock Guardrails at https://aws.amazon.com/bedrock/guardrails/. The evaluation uses a non-public augmentation of ConditionalQA [32].

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Amazon Web Services: Amazon logistics automates electric vehicle design reviews on AWS. https://aws.amazon.com/solutions/case-studies/amazon-logistics-case-study/ (2025). Accessed 01 May 2026
2. An, C., et al.: A neurosymbolic approach to natural language formalization and verification (extended version) (2026). https://arxiv.org/abs/2511.09008
3. Barrett, C., Fontaine, P., Tinelli, C.: The Satisfiability Modulo Theories Library (SMT-LIB). www.SMT-LIB.org (2016)
4. Brewka, G., Eiter, T., Truszczyński, M.: Answer set programming at a glance. Commun. ACM **54**(12), 92–103 (2011). https://doi.org/10.1145/2043174.2043195
5. Callewaert, B., Vandevelde, S., Vennekens, J.: VERUS-LM: a versatile framework for combining LLMs with symbolic reasoning. In: Gebser, M., Inclezan, D., Ricca, F., Carro, M., Truszczynski, M. (eds.) Proceedings 41st International Conference on Logic Programming, ICLP 2025, Rende, Italy, 12–19th September 2025, pp. 47–62. EPTCS (2025). https://doi.org/10.4204/EPTCS.439.5
6. Carbonnelle, P., Vandevelde, S., Vennekens, J., Denecker, M.: Interactive configurator with fo (.) and idp-z3 (2022). arXiv preprint arXiv:2202.00343
7. Casadio, M., et al.: NLP verification: towards a general methodology for certifying robustness. Eur. J. Appl. Math. **37**(1), 180–237 (2026). https://doi.org/10.1017/S0956792525000099
8. Ganguly, D., Iyengar, S., Chaudhary, V., Kalyanaraman, S.: Proof of thought : neurosymbolic program synthesis allows robust and interpretable reasoning. CoRR abs/2409.17270 (2024). https://doi.org/10.48550/ARXIV.2409.17270
9. Haltaufderheide, J., Ranisch, R.: The ethics of chatgpt in medicine and healthcare: a systematic review on large language models (LLMs). npj Digit. Med. **7**(1), (2024). https://doi.org/10.1038/S41746-024-01157-X
10. Han, S., Liu, T., Li, C., Xiong, X., Cohan, A.: HYBRIDMIND: meta selection of natural language and symbolic language for enhanced LLM reasoning. arXiv e-prints arXiv:2409.19381 (2024). https://doi.org/10.48550/arXiv.2409.19381
11. Han, S., et al.: FOLIO: natural language reasoning with first-order logic. In: Al-Onaizan, Y., Bansal, M., Chen, Y. (eds.) Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, 12–16 November 2024, pp. 22017–22031. Association for Computational Linguistics (2024). https://doi.org/10.18653/V1/2024.EMNLP-MAIN.1229

12. Hitzler, P., Sarker, M.K. (eds.): Neuro-Symbolic Artificial Intelligence: The State of the Art, Frontiers in Artificial Intelligence and Applications, vol. 342. IOS Press (2021). https://doi.org/10.3233/FAIA342
13. Hu, X., et al.: Knowledge-centric hallucination detection. In: Al-Onaizan, Y., Bansal, M., Chen, Y.N. (eds.) Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 6953–6975. Association for Computational Linguistics, Miami, Florida, USA (2024). https://doi.org/10.18653/v1/2024.emnlp-main.395, https://aclanthology.org/2024.emnlp-main.395/
14. ICME: AI agents can move money - Lobstar & Wilde proved they can lose it too. https://blog.icme.io/ai-agents-can-move-money-lobstar-wilde-proved-they-can-lose-it-too/ (2026). Accessed 01 May 2026
15. Ishay, A., Yang, Z., Lee, J.: Leveraging large language models to generate answer set programs. In: Marquis, P., Son, T.C., Kern-Isberner, G. (eds.) Proceedings of the 20th International Conference on Principles of Knowledge Representation and Reasoning, KR 2023, Rhodes, Greece, 2–8 September 2023, pp. 374–383 (2023). https://doi.org/10.24963/KR.2023/37
16. Jacovi, A., et al.: The FACTS grounding leaderboard: benchmarking LLMs' ability to ground responses to long-form input. CoRR abs/2501.03200 (2025). https://doi.org/10.48550/ARXIV.2501.03200
17. Jiang, A.Q., et al.: Draft, sketch, and prove: guiding formal theorem provers with informal proofs. In: The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, 1–5 May 2023. OpenReview.net (2023). https://openreview.net/forum?id=SMa9EAovKMC
18. Labs, B.: Bespoke-minicheck-7b (2024). https://huggingface.co/bespokelabs/Bespoke-MiniCheck-7B
19. Levy, M., Jacoby, A., Goldberg, Y.: Same task, more tokens: the impact of input length on the reasoning performance of large language models. In: Ku, L., Martins, A., Srikumar, V. (eds.) Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, 11–16 August 2024, pp. 15339–15353. Association for Computational Linguistics (2024). https://doi.org/10.18653/V1/2024.ACL-LONG.818
20. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks (2021). https://arxiv.org/abs/2005.11401
21. Liu, T., et al.: Logic-of-thought: injecting logic into contexts for full reasoning in large language models. In: Chiruzzo, L., Ritter, A., Wang, L. (eds.) Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pp. 10168–10185. Association for Computational Linguistics, Albuquerque, New Mexico (2025). https://doi.org/10.18653/v1/2025.naacl-long.510, https://aclanthology.org/2025.naacl-long.510/
22. Manakul, P., Liusie, A., Gales, M.J.F.: SelfCheckGPT: zero-resource black-box hallucination detection for generative large language models. In: Bouamor, H., Pino, J., Bali, K. (eds.) Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, 6–10 December 2023, pp. 9004–9017. Association for Computational Linguistics (2023). https://doi.org/10.18653/V1/2023.EMNLP-MAIN.557
23. McCune, W.: Release of prover9. In: Mile High Conference On Quasigroups, Loops and Nonassociative Systems, Denver, Colorado (2005)
24. Merigoux, D., Chataing, N., Protzenko, J.: Catala: a programming language for the law. Proc. ACM Program. Lang. **5**(ICFP), 1–29 (2021). https://doi.org/10.1145/3473582

25. de Moura, L.M., Bjørner, N.S.: Z3: an efficient SMT solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings, pp. 337–340. Lecture Notes in Computer Science, Springer (2008). https://doi.org/10.1007/978-3-540-78800-3_24
26. Olausson, T., et al.: LINC: a neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In: Bouamor, H., Pino, J., Bali, K. (eds.) Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, pp. 5153–5176. Association for Computational Linguistics, Singapore (2023). https://doi.org/10.18653/v1/2023.emnlp-main.313, https://aclanthology.org/2023.emnlp-main.313/
27. Pan, L., Albalak, A., Wang, X., Wang, W.: Logic-LM: empowering large language models with symbolic solvers for faithful logical reasoning. In: Bouamor, H., Pino, J., Bali, K. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 3806–3824. Association for Computational Linguistics, Singapore (2023). https://doi.org/10.18653/v1/2023.findings-emnlp.248, https://aclanthology.org/2023.findings-emnlp.248/
28. PwC Australia: Operationalising responsible AI in education: Automated reasoning and deterministic guardrails on AWS. https://www.pwc.com.au/alliances/amazon-web-services/operationalising-responsible-ai-in-education.html (2026). Accessed 01 May 2026
29. Rajeev, M.A., et al.: Cats confuse reasoning LLM: query agnostic adversarial triggers for reasoning models. CoRR abs/2503.01781 (2025). https://doi.org/10.48550/ARXIV.2503.01781
30. Robinson, J.A., Voronkov, A. (eds.): Handbook of Automated Reasoning (in 2 volumes). Elsevier and MIT Press (2001). https://www.sciencedirect.com/book/9780444508133/handbook-of-automated-reasoning
31. Ryu, H., Kim, G., Lee, H.S., Yang, E.: Divide and translate: compositional first-order logic translation and verification for complex logical reasoning. In: The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, 24–28 April 2025. OpenReview.net (2025). https://openreview.net/forum?id=09FiNmvNMw
32. Sun, H., Cohen, W.W., Salakhutdinov, R.: Conditionalqa: a complex reading comprehension dataset with conditional answers. In: Muresan, S., Nakov, P., Villavicencio, A. (eds.) Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2022, Dublin, Ireland, 22–27 May 2022, pp. 3627–3637. Association for Computational Linguistics (2022). https://doi.org/10.18653/V1/2022.ACL-LONG.253
33. Szegedy, C.: A promising path towards autoformalization and general artificial intelligence. In: Benzmüller, C., Miller, B. (eds.) CICM 2020. LNCS (LNAI), vol. 12236, pp. 3–20. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-53518-6_1
34. Tafjord, O., Dalvi, B., Clark, P.: Proofwriter: generating implications, proofs, and abductive statements over natural language. In: Zong, C., Xia, F., Li, W., Navigli, R. (eds.) Findings of the Association for Computational Linguistics: ACL/IJCNLP 2021, Online Event, 1–6 August 2021, pp. 3621–3634. Findings of ACL, Association for Computational Linguistics (2021). https://doi.org/10.18653/V1/2021.FINDINGS-ACL.317

35. Tang, L., Laban, P., Durrett, G.: Minicheck: efficient fact-checking of LLMs on grounding documents. In: Al-Onaizan, Y., Bansal, M., Chen, Y. (eds.) Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, 12–16 November 2024, pp. 8818–8847. Association for Computational Linguistics (2024). https://doi.org/10.18653/V1/2024.EMNLP-MAIN.499
36. Tian, J., Li, Y., Chen, W., Xiao, L., He, H., Jin, Y.: Diagnosing the first-order logical reasoning ability through logicnli. In: Moens, M., Huang, X., Specia, L., Yih, S.W. (eds.) Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7–11 November, 2021, pp. 3738–3747. Association for Computational Linguistics (2021). https://doi.org/10.18653/V1/2021.EMNLP-MAIN.303
37. Wang, Q., Kaliszyk, C., Urban, J.: First experiments with neural translation of informal to formal mathematics. In: Rabe, F., Farmer, W.M., Passmore, G.O., Youssef, A. (eds.) Intelligent Computer Mathematics - 11th International Conference, CICM 2018, Hagenberg, Austria, 13–17 August 2018, Proceedings. pp. 255–270. Lecture Notes in Computer Science, Springer (2018). https://doi.org/10.1007/978-3-319-96812-4_22
38. Wang, Y., et al.: Factcheck-bench: fine-grained evaluation benchmark for automatic fact-checkers. In: Al-Onaizan, Y., Bansal, M., Chen, Y.N. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2024, pp. 14199–14230. Association for Computational Linguistics, Miami, Florida, USA (2024). https://doi.org/10.18653/v1/2024.findings-emnlp.830, https://aclanthology.org/2024.findings-emnlp.830/
39. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022 (2022). https://doi.org/10.52202/068431-1800, http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html
40. Wu, Y., et al.: Autoformalization with large language models. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9 (2022). https://doi.org/10.52202/068431-2344, http://papers.nips.cc/paper_files/paper/2022/hash/d0c6bc641a56bebee9d985b937307367-Abstract-Conference.html
41. Xu, J., Fei, H., Pan, L., Liu, Q., Lee, M.L., Hsu, W.: Faithful logical reasoning via symbolic chain-of-thought. In: Ku, L.W., Martins, A., Srikumar, V. (eds.) Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 13326–13365. Association for Computational Linguistics, Bangkok, Thailand (2024). https://doi.org/10.18653/v1/2024.acl-long.720
42. Xu, Z., Jain, S., Kankanhalli, M.S.: Hallucination is inevitable: an innate limitation of large language models. CoRR abs/2401.11817 (2024). https://doi.org/10.48550/ARXIV.2401.11817

43. Yang, Z., Ishay, A., Lee, J.: Coupling large language models with logic programming for robust and general reasoning from text. In: Rogers, A., Boyd-Graber, J.L., Okazaki, N. (eds.) Findings of the Association for Computational Linguistics: ACL 2023, Toronto, Canada, 9–14 July 2023, pp. 5186–5219. Findings of ACL, Association for Computational Linguistics (2023). https://doi.org/10.18653/V1/2023.FINDINGS-ACL.321
44. Yao, S., et al.: Tree of thoughts: Deliberate problem solving with large language models. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, 10–16 December 2023 (2023). https://doi.org/10.52202/075280-0517, http://papers.nips.cc/paper_files/paper/2023/hash/271db9922b8d1f4dd7aaef84ed5ac703-Abstract-Conference.html
45. Ye, X., Chen, Q., Dillig, I., Durrett, G.: Satlm: satisfiability-aided language models using declarative prompting. In: Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, 10–16 December 2023 (2023). https://doi.org/10.52202/075280-1974, http://papers.nips.cc/paper_files/paper/2023/hash/8e9c7d4a48bdac81a58f983a64aaf42b-Abstract-Conference.html

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (http://creativecommons.org/licenses/by/4.0/), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

